C++ Programming Concepts : SOLID Principles and Design Patterns

LSEG Technology

26th Feb 2026

SOLID Principles

- **Software always change**
- Software should be written such that it supports changes.
- For a good Object-Oriented Design of a Software, it should be easy to
  - Understand
  - Maintain
  - Extend
- Many Principles Guide us to design quality software.
  - SOLID
  - GRASP
  - DRY

SOLID:

- First introduced by Uncle Bob (Robert C.Martin) in his 2000 paper Design Principles and Design Patterns.
- Acronym was introduced later, around 2004, by Michael Feathers

What is SOLID?

- S –Single Responsibility Principle (SRP)
- O – Open Closed Principle (OCP)
- L – Liskov Substitution Principle (LSP)
- I – Interface Segregation Principle (ISP)
- D – Dependency Inversion Principle (DIP)

What is SOLID?

S –Single Responsibility Principle (SRP)

- Each software component should have only one reason to change

O – Open Closed Principle (OCP)

L – Liskov Substitution Principle (LSP)

I – Interface Segregation Principle (ISP)

D – Dependency Inversion Principle (DIP)

S - for Single Responsibility Principle (SRP)

Each Software Component should have One & Only One Responsibility.

OR, As Uncle Bob Says:

Each Software Component should have One & Only One Reason To Change

Image courtesy of Derick Bailey

More responsibilities/reasons to change → More frequent changes → More possibility of Bugs → Not easy to Maintain

S - for Single Responsibility Principle (SRP)

```
class  Student
{
private:
    int id;
    string name;
    int age;
    string email;
    int marks[9];

    string dburl;
    string emailpassword;
public:
    Student(int id, string name, int age, string email): id(id), name(name), age(age), email(email)
    {


    }
    string getName() { return name; }
    int getId() { return id;  }
    int getAge() { return age; }

    void save() {
        cout << "Save called" << endl;
        //create db connection
        //code to save to DB
    }

    void sendEmail(string toemail, string content)
    {
        cout << "Send email called" << endl;
        //create smtp connection
        //send email
    }
};

int main()
{
    Student student(1,"John",23, "john@gmail.com");

    student.save();
    student.sendEmail("warne@gmail.com", "Hi, How are you");


}
```

What are the reasons for changes to this class??

1. Changing student information

2. Changing database backend

3. Changing email sending options

4. …

S - for Single Responsibility Principle (SRP)

```
class DBConnection
{
    string dburl;
public:
    DBConnection()
    {
        //create the db connection
    }

    void save()
    {
        cout << "Save called" << endl;
        //create db connection
        //code to save to DB
    }

};

class EmailSender
{
    string email;
    string emailpassword;
public:
    void sendEmail(string toemail, string content)
    {
        cout << "Send email called" << endl;
        //create smtp connection
        //send email
    }
};
```

```
class  Student
{
private:
    int id;
    string name;
    int age;

    int marks[9];

    DBConnection dbconnection;
    EmailSender emailSender;


public:
    Student(int id, string name, int age, string email): id(id), name(name), age(age)
    {

    }
    string getName() { return name; }
    int getId() { return id;  }
    int getAge() { return age; }

    void save() {
        dbconnection.save();
    }

    void sendEmail(string toemail, string content)
    {
        emailSender.sendEmail(toemail, content);
    }

};
```

Classes have only 1 responsibility to each.

Student: keeps track of student information

DBConnection: handles DB queries

EmailSender: handle email related stuff

S - for Single Responsibility Principle (SRP)

For Better Adherence to SRP

More Related Components within

High Cohesion

(The degree to which the various parts of a software components are related)

Loose Coupling

(The level of inter dependency between various software components)

Inter Dependency with Outside Low

S - for Single Responsibility Principle (SRP) - Tips

- Prevent Antipattern of God Object (1 Class doing all): (the opposite of SRP)
- Always try High Cohesion and Loose Coupling.
- But, Prevent Needless Complexity:
  - Group responsibilities/reasons to change in a related way.
  - Don’t try to create classes/separate modules for simplest levels.
- Always separate business logic and persistent logic.
- Use Facade, DAO or Proxy patterns to separate responsibilities.

What is SOLID?

S –Single Responsibility Principle (SRP)

- Each software component should have only one reason to change

O – Open Closed Principle (OCP)

- Software components should be closed for modification, but open for extension

L – Liskov Substitution Principle (LSP)

I – Interface Segregation Principle (ISP)

D – Dependency Inversion Principle (DIP)

O - for Open Closed Principle (OCP)

Software Entities (classes, modules, functions, etc.) should be OPEN for EXTENSION but CLOSED for MODIFICATION.

To add New features, we should not modify existing code.

- Abstraction is the Key.
- Model the behavior through an abstraction (interfaces/abstract classes) Use concrete classes for extension
- Testing is easy as we don’t touch already available code

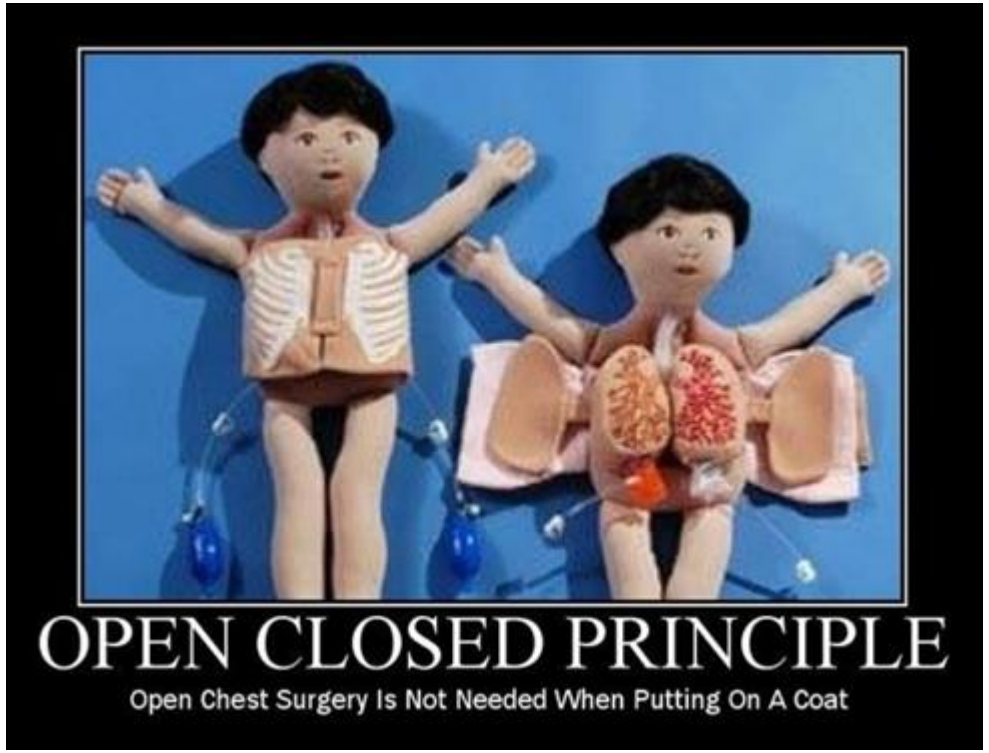


Image courtesy of Derick Bailey

The idea was 1st given by Bertrand Meyer, in 1988.

O - for Open Closed Principle (OCP)

```
enum ShapeType
{
    Circle,
    Square,
    Pentagon
};
class ShapeDrawer
{
    void drawCircle() { cout << "Draw Circle" << endl; }
    void drawSquare(){ cout << "Draw Square" << endl; }
    void drawPentagon(){ cout << "Draw Pentagon" << endl; }
public:
    void draw(ShapeType s)
    {
        if (s == Circle)
        {
            drawCircle();
        }
        else if (s = Square)
        {
            drawSquare();
        }
        else if (s = Pentagon)
        {
            drawPentagon();
        }
    }
};


int main()
{
    ShapeDrawer shapedrawer;
    shapedrawer.draw(Circle);
    shapedrawer.draw(Pentagon);

}
```

- **Fragile Code**
- **Recompilation**
- **Testing Burden**
- **Scalability**
- **Violation of Single Point of Change**

O - for Open Closed Principle (OCP)

```
class Shape
{
public:
    virtual void draw() = 0;
};

class Circle : public Shape
{
public:
    virtual void draw()
    {
        cout << "Draw Circle" << endl;
    }
};

class Square : public Shape
{
public:
    virtual void draw()
    {
        cout << "Draw Square" << endl;
    }
};

class Pentagon : public Shape
{
public:
    virtual void draw()
    {
        cout << "Draw Pentagon" << endl;
    }
};
```

```
int main()
{
    Shape* s = new Circle();
    s->draw();
    s = new Square();
    s->draw();
    s = new Pentagon();
    s->draw();
}
```

O - for Open Closed Principle (OCP) -Tips

- Always use abstractions.
- If a single change in 1 program, causes a set of changes in other modules, OCP is violated.
- Functions with if blocks or switch statements checking type of sub class objects, are violating this rule.
- No program can be 100% closed. Always check the probability of different changes and apply OCP for most frequent changes.
- Heuristics related to OCP in OOD:
  - Make all member variables private
  - Don't use global variables
  - Prevent RTTI (run time type identification) including dynamic_cast and static_cast that violate OCP.
- Don’t follow OCP blindly. If to fix a simple bug in current code, don’t try OCP. But, may be to change code for a bug that is due to bad design.

What is SOLID?

S –Single Responsibility Principle (SRP)

- Each software component should have only one reason to change

O – Open Closed Principle (OCP)

- Software components should be closed for modification, but open for extension

L – Liskov Substitution Principle (LSP)

- Objects should be replaceable with their subtypes without affecting the correctness of the program

I – Interface Segregation Principle (ISP)

D – Dependency Inversion Principle (DIP)

L – Liskov Substitution Principle (LSP)

By Barbara Liskov in 1988 originally as

"What is wanted here is something like the following substitution property: If for each object o1 of type S there is an object o2 of type T such that for all programs P defined in terms of T, the behavior of P is unchanged when o1 is substituted for o2 then S is a subtype of T."

Functions that use pointers or references to base classes must be able to use objects of derived classes without knowing it.

In simple words,

- If any subclass does nothing or do not have a sufficient overriding of any method in the super class, then it violates this principle.
- All methods in super class must have a meaning at its sub class.

Inheritance is not merely a IS-A relationship. We should consider behaviors we Need.

Image courtesy of Derick Bailey

L – Liskov Substitution Principle (LSP)

```
class Bird
{

public:
    virtual void fly() =0;
};

class Parrot: public Bird
{
    //this is okay
    void fly()
    {
        cout << "Bird flies" << endl;
    }
};

class Ostrich : public Bird
{
    //this is wrong. Ostrich cant fly :(
    void fly()
    {
        cerr << "Ostrich cannot fly" << endl;
    }
};

int main()
{
    Bird* b = new Parrot();
    b->fly();
    b = new Ostrich();
    b->fly();
}
```

L – Liskov Substitution Principle (LSP)

```
class Bird
{

public:

};

class FlyingBird : public Bird
{
public:
    void fly()
    {
        cout << "Bird flies" << endl;
    }
};

class Parrot: public FlyingBird
{
    //Parrot can fly, so it inherits from FlyingBird
};

class Ostrich : public Bird
{
    //Ostrich cannot fly, so it inherits directly from Bird
};

int main()
{
    FlyingBird* b = new Parrot();
    b->fly();
    Bird* nb = new Ostrich();
    //nb->fly();
}
```

L – Liskov Substitution Principle (LSP) - Tips

- Prevent matching the real world (ISA relationship) always.
- Always think about what is the responsibility of the class and what functions and behaviors you have in a class in creating subclasses.

- How to solve the issue?
  - Break the hierarchy into more granular level.
  - Restructure code such that related class do the related functionality.

What is SOLID?

S –Single Responsibility Principle (SRP)

- Each software component should have only one reason to change

O – Open Closed Principle (OCP)

- Software components should be closed for modification, but open for extension

L – Liskov Substitution Principle (LSP)

- Objects should be replaceable with their subtypes without affecting the correctness of the program

I – Interface Segregation Principle (ISP)

- No client should be forced to depend on interfaces it does not use

D – Dependency Inversion Principle (DIP)

I – Interface Segregation Principle (ISP)

Clients should not be forced to depend upon interfaces that they do not use.

Separate "fat interfaces" into abstract base classes that break unwanted coupling between clients.

Fat interfaces :

- a lot of method definitions in it.
- client who use it have to override all of them whether or not he need them

Use thin or small interfaces so that their reusability is high.

Image courtesy of Derick Bailey

I – Interface Segregation Principle (ISP)

```
class  MultiFunctionMachine
{
public:
    virtual void photocopy() = 0;
    virtual void scan() = 0;
    virtual void print() = 0;
};

class MultiFunctionPhotocopyMachine : public MultiFunctionMachine
{
public:
    virtual void photocopy()
    {
        cout << "MultiFunction Photocopy Success" << endl;
    }
    virtual void scan()
    {
        cout << "MultiFunction Scan Success" << endl;
    }
    virtual void print()
    {
        cout << "MultiFunction Print Success" << endl;
    }
};

class Printer : public MultiFunctionMachine
{
public:
    virtual void photocopy()
    {
        cout << "Printer : Photocopy Not Implemented" << endl;
    }
    virtual void scan()
    {
        cout << "Printer : Scan Not Implemented" << endl;
    }
    virtual void print()
    {
        cout << "Printer Print Success" << endl;
    }
};
```

```
class PrintAndCopyMachine : public MultiFunctionMachine
{
public:
    virtual void photocopy()
    {
        cout << "PrintAndCopyMachine Photocopy Success" << endl;
    }

    virtual void scan()
    {
        cout << "PrintAndCopyMachine : Scan Not Implemented" << endl;
    }
    virtual void print()
    {
        cout << "PrintAndCopyMachine Print Success" << endl;
    }
};
int main()
{
    MultiFunctionMachine* m1 = new MultiFunctionPhotocopyMachine();
    m1->print();
    m1->scan();
    MultiFunctionMachine* m2 = new PrintAndCopyMachine();
    m2->print();
    m2->scan();
    MultiFunctionMachine* m3 = new Printer();
    m3->print();
    m3->scan();

}
```

I – Interface Segregation Principle (ISP)

```
class  IPhotocopy
{
public:
    virtual void photocopy() = 0;
};
class IPrint
{
public:
    virtual void print() = 0;
};
class IScan
{
public:
    virtual void scan() = 0;
};

class MultiFunctionPhotocopyMachine : public IPhotocopy,IPrint, IScan
{
public:

    virtual void photocopy() override
    {
        cout << "MultiFunction Photocopy Success" << endl;
    }
    virtual void scan() override
    {
        cout << "MultiFunction Scan Success" << endl;
    }
    virtual void print() override
    {
        cout << "MultiFunction Print Success" << endl;
    }
};
```

```
class Printer : public IPrint
{
public:
    virtual void print() override
    {
        cout << "Printer Print Success" << endl;
    }
};

class PrintAndCopyMachine : public IPhotocopy, IPrint
{
public:
    virtual void photocopy() override
    {
        cout << "PrintAndCopyMachine Photocopy Success" << endl;
    }


    virtual void print() override
    {
        cout << "PrintAndCopyMachine Print Success" << endl;
    }
};
int main()
{
    MultiFunctionPhotocopyMachine m1;
    m1.print();
    m1.scan();
    PrintAndCopyMachine m2;
    m2.print();
    //m2.scan();
    Printer m3;
    m3.print();
    //m3.scan();

}
```

I – Interface Segregation Principle (ISP) - Tips

- Check for unimplemented methods in concrete classes. ISP violated. : Break interfaces to avoid unused methods.
- Check for FAT interfaces
- Check for interfaces with Low Cohesion : unrelated methods
- If interfaces follow SRP→ ISP too is preserved.

What is SOLID?

S –Single Responsibility Principle (SRP)

- Each software component should have only one reason to change

O – Open Closed Principle (OCP)

- Software components should be closed for modification, but open for extension

L – Liskov Substitution Principle (LSP)

- Objects should be replaceable with their subtypes without affecting the correctness of the program

I – Interface Segregation Principle (ISP)

- No client should be forced to depend on interfaces it does not use

D – Dependency Inversion Principle (DIP)

- High level modules should not depend on low level modules. Both should depend on abstractions.
- Abstractions should not depend upon details. Details should depend upon abstractions.

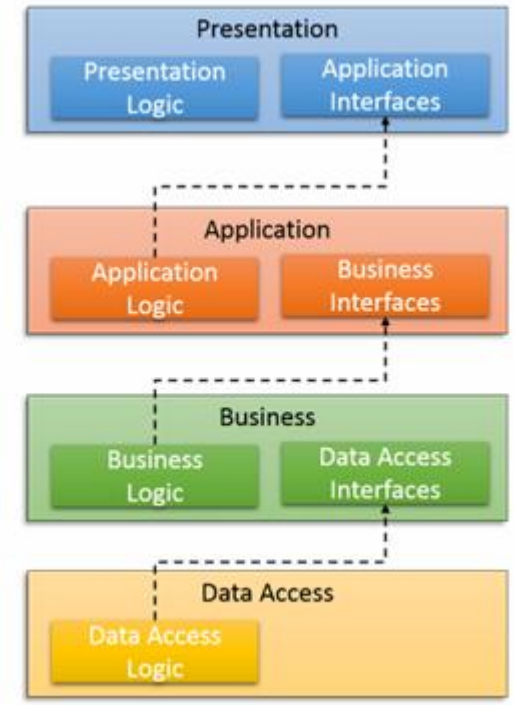
D – Dependency Inversion Principle (DIP)

A. **High level modules should not depend upon low level modules. Both should depend upon abstractions.**

A. **B. Abstractions should not depend upon details. Details should depend upon abstractions.**

High level modules:
More closer to business logic

Low level modules:
More closer to low level implementation (databases/HW interfaces etc.)



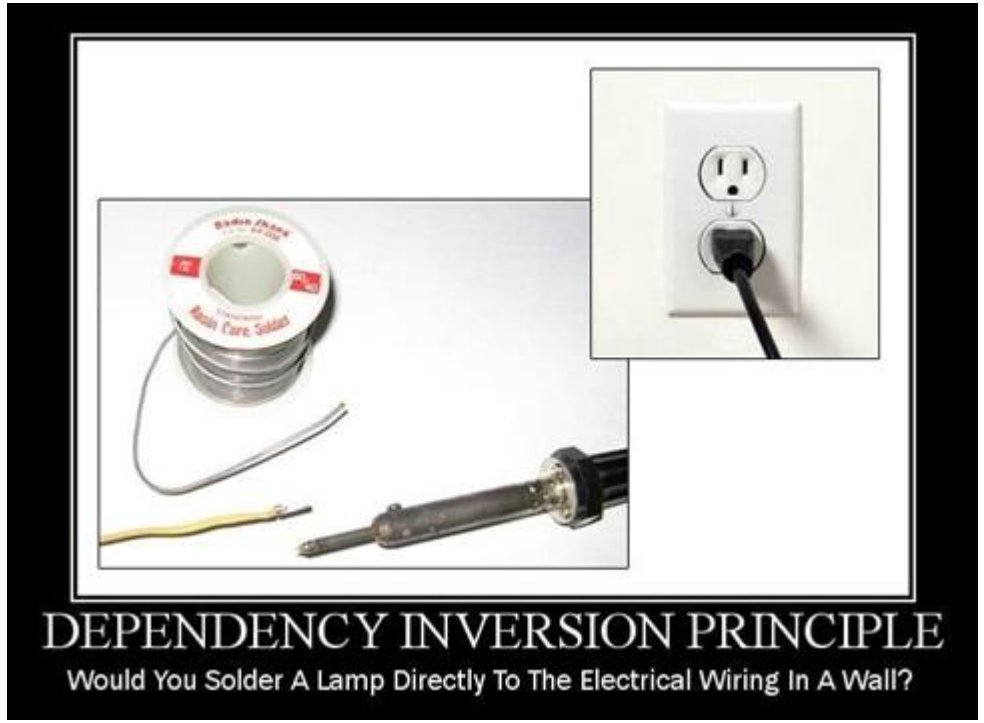


Image courtesy of Derick Bailey

D – Dependency Inversion Principle (DIP)

```
class CustomerDataAccess
{
public:
    string getCustomerName(int id) {
        return "Customer Name Derived from DB";
    }
};

class CustomerBusinessLogic
{
    CustomerDataAccess dataAccess;
public:
    string getCustomerName(int id)
    {
        return dataAccess.getCustomerName(id);
    }
};


int main()
{
    CustomerBusinessLogic customer;
    string name = customer.getCustomerName(1);
    cout << name << endl;

}
```

D – Dependency Inversion Principle (DIP)

```
class ICustomerDataAccess
{
public:
    virtual string getCustomerName(int id) = 0;
};

class CustomerDataAccess : public ICustomerDataAccess
{
public:
    virtual string getCustomerName(int id) {
        return "Customer Name Derived from DB";
    }
};


class CustomerBusinessLogic
{
    ICustomerDataAccess* dataAccess = new CustomerDataAccess();
public:
    string getCustomerName(int id)
    {
        return dataAccess->getCustomerName(id);
    }
};


int main()
{
    CustomerBusinessLogic customer;
    string name = customer.getCustomerName(1);
    cout << name << endl;

}
```

What is SOLID?

S –Single Responsibility Principle (SRP)

- Each software component should have only one reason to change

O – Open Closed Principle (OCP)

- Software components should be closed for modification, but open for extension

L – Liskov Substitution Principle (LSP)

- Objects should be replaceable with their subtypes without affecting the correctness of the program

I – Interface Segregation Principle (ISP)

- No client should be forced to depend on interfaces it does not use

D – Dependency Inversion Principle (DIP)

- High level modules should not depend on low level modules. Both should depend on abstractions.
- Abstractions should not depend upon details. Details should depend upon abstractions.

Conclusion

- If OOP is like grammar, OOD is actually writing an essay with the grammar.

- SOLID principles helps to design quality software which easy to maintain, extend and understand.

- In addition to SOLID, there are several other principles that are used in OOP such as
  - GRASP : 9 fundamental principles in object design and responsibility assignment
    - Information expert.
    - Creator.
    - Low coupling.
    - Protected variations.
    - Indirection.
    - Polymorphism.
    - High cohesion.
    - Pure fabrication.

  - DRY : Don’t Repeat Yourself
    - Every piece of knowledge must have a single, unambiguous, authoritative representation within a system

Design Patterns

Design Patterns

- What are design patterns?
- Benefits of design patterns
- Categorization of design patterns
- Examples

What is a design pattern?

- Typical solutions to commonly occurring problems in software design.
- Patterns are like pre-made blueprints that you can customize to solve a recurring problem in your code.
- Pattern is a general concept, not a specific piece of code.
- Does patterns and algorithms are same or different?
- The concept of design patterns originated in other fields but with the publication of "*Elements of Reusable Object-Oriented Software*" by Gang of Four in 1994, concept has widely adopted by the developers.

What contains in a design pattern?

Patterns are formally described and mostly consists with below information.

- Pattern name
- The problem: Explains the problem and the context
- The solution: Describes the elements that makeup the design, their relationships, responsibilities, and collaborations.
- Consequences: Results and the trade-offs of applying the pattern
- Code example

Why design pattern?

- Solutions defined in the design patterns are obtained by trial and error over a substantial period.

  So, you can use those without reinventing the wheel.

- Increases the readability, maintainability, and speed up the development process.

- It is a common language.

Categorization of Patterns

- Around 26 patterns are exist.
- Can be categorized into 3 main types

- Creational design patterns
  - About class instantiation
  - Eg: Singleton, Factory method, Abstract factory, Builder

- Structural design patterns
  - About class and object composition
  - Eg: Adapter, Bridge, Composite, Decorator, Facade

- Behavioral design patterns
  - About class's objects communication
  - Eg: Chain of responsibility, Command, Interpreter, Mediator, Observer

Creational Design Patterns

Purpose:
Manage object creation in a flexible and efficient way.
Avoid hard-coded creation logic and tight coupling.

Key Focus Areas:

- How objects are created
- Controlling creation process
- Reducing complexity in instantiation

When to Use:

- When object creation is complicated
- When objects need to be decoupled from their creation logic

Popular Creational Patterns:

- **Singleton** – Ensures only one instance exists
- **Factory Method** – Creates objects without specifying exact class
- **Abstract Factory** – Creates families of related objects
- **Builder** – Builds complex objects step-by-step
- **Prototype** – Creates new objects by cloning existing ones

Structural Design Patterns

Purpose:
Describe how classes and objects are combined to form larger structures.

Key Focus Areas:

- Composition over inheritance
- Flexible and efficient structure organization
- Reducing dependencies between components

When to Use:

- When system structure becomes complex
- When changes should not break existing code

Popular Structural Patterns:

- **Adapter** – Makes incompatible interfaces work together
- **Decorator** – Adds new behavior without modifying the original class
- **Facade** – Simplifies complex systems through a single interface
- **Composite** – Treats individual objects and groups uniformly
- **Proxy** – Provides a placeholder or control access to another object
- **Bridge** – Separates abstraction from implementation

Behavioral Design Patterns

Purpose:
Define how objects interact and communicate.

Key Focus Areas:

- Object collaboration
- Distribution of responsibilities
- Making communication flexible

When to Use:

- When communication logic is complex
- When behavior needs to be interchangeable

Popular Behavioral Patterns:

- **Observer** – Notifies multiple objects automatically on state change
- **Strategy** – Allows switching algorithms at runtime
- **Command** – Encapsulates operations as objects
- **Iterator** – Enables traversal over collections
- **Mediator** – Simplifies communication through a central coordinator
- **State** – Changes behavior based on internal state
- **Chain of Responsibility** – Passes request along a chain of handlers

Singleton (Creational)

- Only one instance of an object is created
- Make the default constructor private
- Create static creation method

```
class Student
{
public:
    Student(const std::string& name)
        : m_name(name)
    {
    }

    ~Student() {}
private:
    const std::string m_name;
};

class StudentRegistry
{
public:
    static StudentRegistry& get() { return m_reg; }

    Student* find(const std::string name) { return nullptr; /*return student found*/ }
    void add(Student* student) {}

private:
    StudentRegistry() {}
    ~StudentRegistry() {}

    static StudentRegistry m_reg;
};

StudentRegistry StudentRegistry::m_reg;

int main()
{
    auto& reg = StudentRegistry::get();
    auto student = reg.find("Student1");

    std::cin.get();
}
```

Factory (Creational)

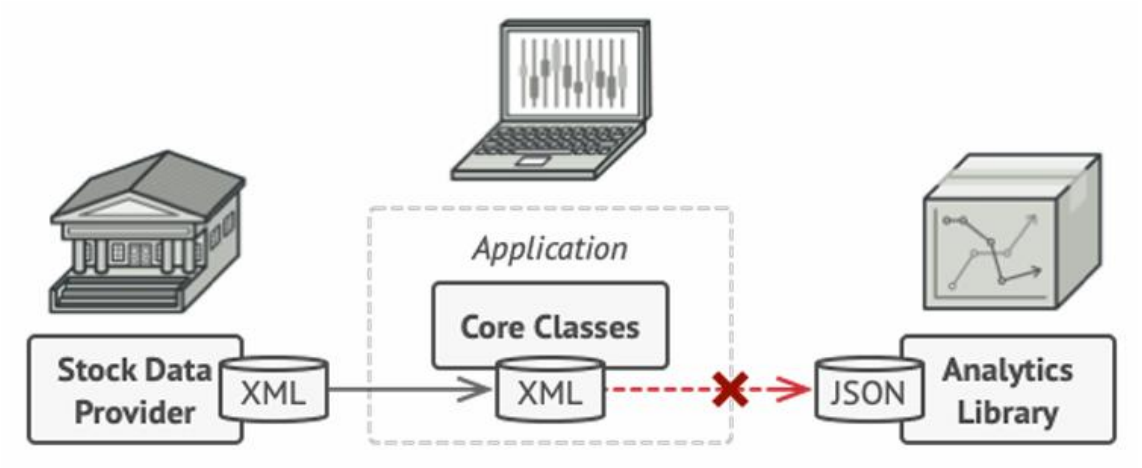
- Create objects without specifying the exact class to create

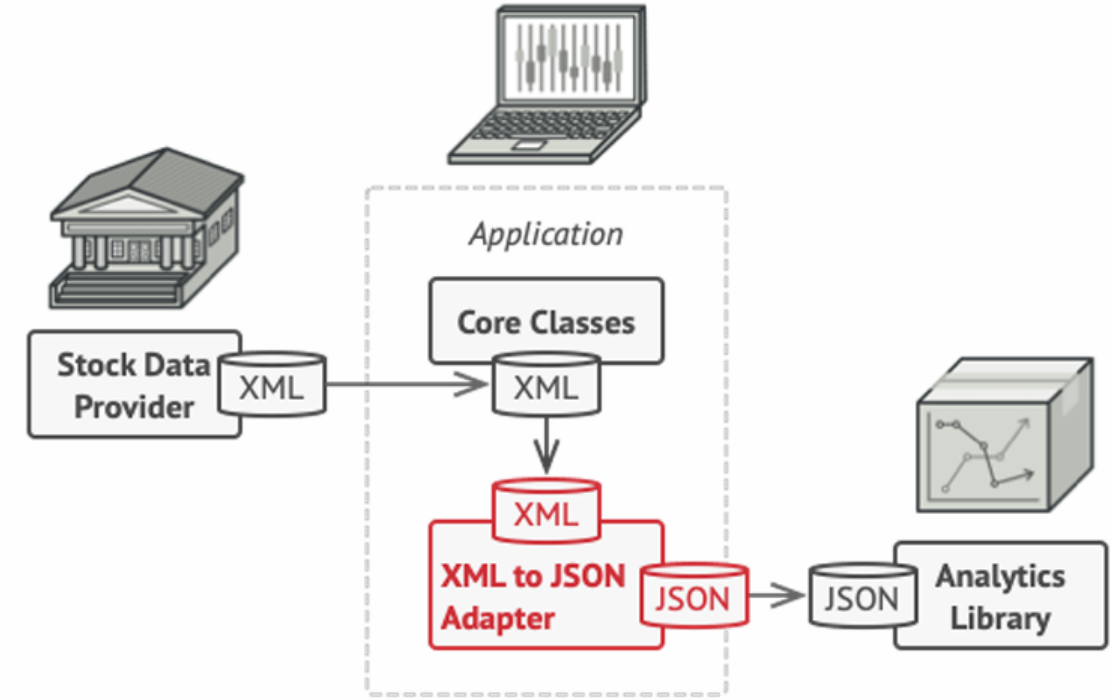
```cpp
class Employee
{
public:
    Employee() {}
    virtual ~Employee() {}

    virtual void calculateSalary() = 0;
};

class FullTimeEmployee : public Employee
{
public:
    FullTimeEmployee() {}
    virtual ~FullTimeEmployee() {}

    void calculateSalary() override /*final also available*/
    {
        std::cout << "FullTimeEmployee - Salary calculation logic" << std::endl;
    }
};

class PartTimeEmployee : public Employee
{
public:
    PartTimeEmployee() {}
    virtual ~PartTimeEmployee() {}

    void calculateSalary() override /*final also available*/
    {
        std::cout << "PartTimeEmployee - Salary calculation logic" << std::endl;
    }
};
```

```cpp
class BaseEmployeeFactory
{
public:
    virtual ~BaseEmployeeFactory() = default;
    virtual std::unique_ptr<Employee> createEmployee(std::string type) = 0;
};

class EmployeeFactory : public BaseEmployeeFactory
{
public:
    std::unique_ptr<Employee> createEmployee(std::string type) override
    {
        if (type == "FullTime")
        {
            return std::make_unique<FullTimeEmployee>();
        }
        else if (type == "PartTime")
        {
            return std::make_unique<PartTimeEmployee>();
        }
        else
        {
            std::cerr << "Unknown type" << std::endl;
            return nullptr;
        }
    }
};

int main()
{
    std::list<std::unique_ptr<Employee>> employees;
    std::unique_ptr<BaseEmployeeFactory> factory = std::make_unique<EmployeeFactory>();
    employees.push_back(std::move(factory->createEmployee("FullTime")));
    employees.push_back(std::move(factory->createEmployee("PartTime")));

    for (auto& employee : employees)
    {
        employee->calculateSalary();
    }

    std::cin.get();
}
```

Adapter (Structural)

- Allows for two incompatible classes to work together
- Wrapping an interface around one of the existing classes





Adapter (Structural)

```cpp
class Message
{
public:
    Message() {}
    virtual ~Message() {}
};

class Encoder
{
public:
    Encoder() {}
    virtual ~Encoder() {}

    virtual const std::string encode(Message& msg) = 0;
};

class JsonEncoder : public Encoder
{
public:
    JsonEncoder() {}
    virtual ~JsonEncoder() {}

    const std::string encode(Message& msg) override /*final also available*/
    {
        std::cout << "JsonEncoder::encode" << std::endl;
        return "JsonEncoder - encoded buffer";
    }
};

class OldXMLEncoder
{
public:
    OldXMLEncoder() {}
    virtual ~OldXMLEncoder() {}

    const void encode(Message& msg, std::string& encoded)
    {
        std::cout << "OldXMLEncoder::encode" << std::endl;
        encoded = "OldXMLEncoder - encoded buffer";
    }
};
```

```cpp
class XMLEncoder : public Encoder
{
public:
    XMLEncoder() {}
    virtual ~XMLEncoder() {}

    const std::string encode(Message& msg) override /*final also available*/
    {
        std::cout << "XMLEncoder::encode" << std::endl;
        std::string encodedBuffer;
        m_oldEncoder.encode(msg, encodedBuffer);
        return encodedBuffer;
    }

private:
    OldXMLEncoder m_oldEncoder;
};


int main()
{
    Message msg;
    std::unique_ptr<Encoder> e1 = std::make_unique<JsonEncoder>();
    std::cout << e1->encode(msg) << std::endl;

    std::cout << std::endl;
    std::unique_ptr<Encoder> e2 = std::make_unique<XMLEncoder>();
    std::cout << e2->encode(msg) << std::endl;

    std::cin.get();
}
```

Observer (Behavior)

- Publish/subscribe pattern

```cpp
class Socket;
class SocketObserver
{
public:
    SocketObserver() {}
    virtual ~SocketObserver() {}
    virtual void update(Socket& socket, const std::string data) = 0;
    //Instead of update can use meaningful name like onData, onRecvData, etc ...
};

class Socket
{
public:
    Socket() {}
    virtual ~Socket() {}
    void onData(const std::string data)
    {
        notifyObservers(data);
    }

    void registerObserver(std::shared_ptr<SocketObserver> observer)
    {
        m_observers.push_back(observer);
    }

private:
    void notifyObservers(const std::string data)
    {
        for (auto observer : m_observers)
        {
            observer->update(*this, data);
        }
    }
    std::list<std::shared_ptr<SocketObserver>> m_observers;
};
```

```cpp
class WindowObserver : public SocketObserver
{
public:
    WindowObserver() {}
    virtual ~WindowObserver() {}
    void update(Socket& socket, const std::string data) override
    {
        std::cout << "Data received by WindowObserver. Data:" << data << std::endl;
    }
};

class ContainerObserver : public SocketObserver
{
public:
    ContainerObserver() {}
    virtual ~ContainerObserver() {}
    void update(Socket& socket, const std::string data) override
    {
        std::cout << "Data received by ContainerObserver. Data:" << data << std::endl;
    }
};

int main()
{
    std::unique_ptr<Socket> socket = std::make_unique<Socket>();
    std::shared_ptr<SocketObserver> observer1 = std::make_shared<WindowObserver>();
    std::shared_ptr<SocketObserver> observer2 = std::make_shared<ContainerObserver>();
    socket->registerObserver(observer1);
    socket->registerObserver(observer2);

    socket->onData("Test data");

    std::cin.get();
}
```

Thank You !